

LECTURE # 07

STACK:

•) A Stack is a LIFO (Last In, First Out) Data structure.

•) The last element you put in will be the first one to come out.

•) Eg: plates on each other, you can only take plate from top & can only place at top. So the last plate you put will be the first one to be taken.

* Key Operations:

① push(x):

Insert element at top.

② pop():

Remove element from top.

③ peek():

See top element

④ display()

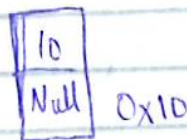
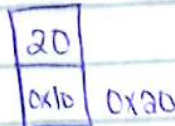
⑤ isfull()

⑥ isempty()

* Why Stack :

- 1) We can reverse strings easily.
- 2) We can undo & redo in text editors.
- 3) Recursion uses stack memory.
- 4) Expression conversion, Infix $\rightarrow$ Postfix
- 5) Parenthesis balancing.

* Diagram :



* Structure of a Node class:

```
class Node  
{
```

```
    public:
```

```
        int data;
```

```
        Node* next;
```

```
        Node(int v): data(v), next(NULL) {}
```

```
};
```

* Structure of Stack class :

```
class Stack
```

```
{  
    private:  
        Node* top;  
        int count;
```

```
public:
```

```
    stack()
```

```
{
```

```
    top = NULL;
```

```
    count = 0;
```

```
}
```

```
    bool isempty() { }
```

```
    bool isfull() { } // array based
```

```
    void push();
```

```
    void pop();
```

```
    void peek();
```

```
    void display();
```

```
    void clear();
```

```
    int getsize(); // node based
```

```
};
```

// Code :

```
#include <iostream>
using namespace std;
```

```
class Node
```

```
{
```

```
public:
```

```
Node* next;
```

```
int data;
```

```
Node(int v): data(v), next(NULL) {}
```

```
};
```

```
class Stack
```

```
{
```

```
private:
```

```
Node* top;
```

```
int count;
```

```
// for arr based
```

```
// int top;
```

```
// int count;
```

```
// int* arr;
```

```
public:
```

```
Stack(): top(NULL), count(0) {}
```

```
// for arr based
```

```
// Stack(int s): top=-1, count(s),
```

```
// { arr = new int[count]; }
```

```
bool isEmpty() { return top == NULL; }
```

```
// for arr based:
```

```
// bool isEmpty() { return top == -1; }
```

```
// bool isFull() { return top == size-1; }
```

```

void push(int val)
{
    Node* newnode = new Node(val);
    newnode->next = top;
    top = newnode;
    count++;
}

```

```

// for array based
// void push(int val)
// {
//     if (isfull()) { cout << "full array"; }
//     else
//     {
//         arr[++top] = v;
//     }
// }

```

```

void pop()
{
    if (isempty())
    {
        cout << "Empty"; return;
    }
    else
    {
        Node* temp = top;
        top = top->next;
        delete temp;
        count--;
        cout << "Deleted";
    }
}

```

```

// for array based:
// void pop()
// {
//     if (isEmpty())
//     {
//         cout << "empty";
//     }
//     else
//     {
//         arr[top--];
//         cout << "deleted";
//     }
// }

```

```

Void peek()
{
    if (isEmpty())
    {
        cout << "empty";
    }
    else
    {
        cout << arr[top];
    }
}

```

// array based

```

Void peek()
{
    if (isEmpty()) { return; }
    cout << arr[top];
}

```

```
void display()
{
    if (isempty()) { return; }

```

```

Node * curr = top;
while (curr != NULL)
{
    cout << curr->data;
}

```

// for arr based

```

void display()
{
    if (isempty()) { return; }

    for (int i = 0; i < count; i++)
    {
        cout << arr[i];
    }
}

```

★ Main Function:

```

int main()
{
    Stack S;

    // Call any function;

    return 0;
}

```

Stack (All-Questions)

(Question # 01)

```
class Stack
```

```
{
```

```
private:
```

```
int* arr;
```

```
int top;
```

```
int cap;
```

```
// data member
```

```
public:
```

```
Stack(int s){
```

```
// constructor
```

```
{
```

```
cap = s;
```

```
arr = new int[cap];
```

```
top = -1;
```

```
// initialize
```

```
}
```

```
~Stack()
```

```
// destructor
```

```
{
```

```
delete[] arr;
```

```
}
```

```
void push(int x)
```

```
// insert element
```

```
{
```

```
if (top == cap - 1)
```

```
// if cap is full
```

```
{
```

```
return;
```

```
}
```

```
top++;
```

```
arr[top] = x
```

```
// increment top
```

```
// assign top index new
```

```
}
```

```

void pop()
{
    if (top == -1)
    {
        return;
    }
    prompt element; top--; // delete last entered element
}

void peek()
{
    if (top == -1)
    {
        return;
    }
    prompt element; // cout element
}

bool is_empty()
{
    return top == -1; // checking emptiness.
}

void display()
{
    if (top == -1)
    {
        return;
    }
    for (int i = top; i >= 0; i--) // run from last to 1st
    {
        prompt elements; // cout elements
    }
}

```

Question # 02

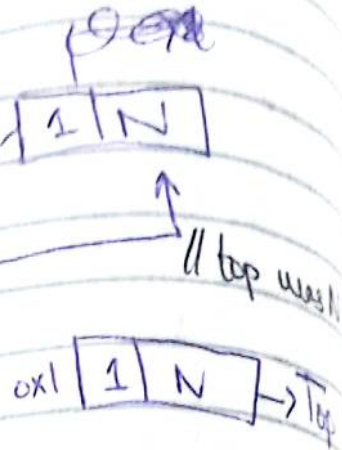
// explained in basic operations of Stack
(DRY-RUN)

•) Suppose: user push 1:

Node* newnode = new Node(1);

newnode->next = top;

top = newnode;

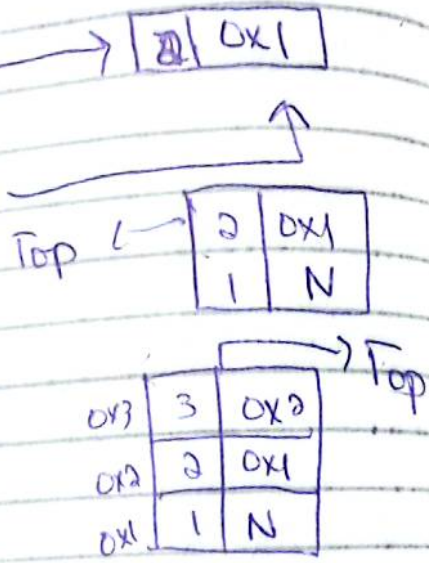


user push 2:

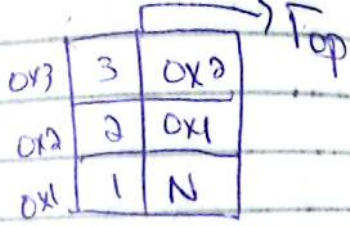
Node* newnode;

newnode->next = top;

top = newnode;



•) user push 3:



•) Now: peek:

if (top == NULL) X top = 0x3

cout << top->data; 3

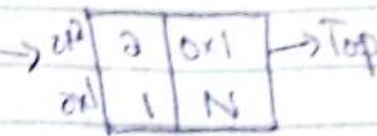
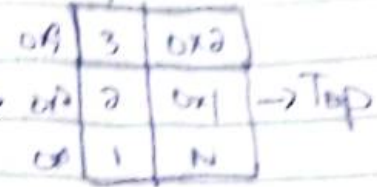
Now Pop:

if (top == Null) X top = 0/3

Node t = top;

top = top -> next;

delete t;



Now display:

if (top == N) X

Node t = top;



while (t != N) ✓

cout << t->data;

t = t->next;

2

0x1

while (t != N) ✓

{

cout << t->data;

t = t->next;

}

1

t = N

while (t != N) X t = N

Question #03

// Remaining part is same

```

void insertatbottom(int x)
{
    if (isEmpty()) { return; }
    int t = pop();
    insertatbottom(x);
    push(t);
}

```

// if no element
// pop into t
// recursing
// push temp

```

void reverse()
{
    if (isEmpty()) { return; }
    int t = pop();
    reverse();
    insertatbottom(t);
}

```

// reverse func
// if no element
// pop into t
// recursing
// call func

(DRY - RUN)

Suppose: our stack is :

3	0x1	-> top
2	0x1	
1	N	

reverse: (a)

if (—) X

```

int t = pop();    t = 2
reverse();

```

reverse: (b)

if (—) X

```

int t = pop();    t = 1
reverse();

```

reverse: (c)
 if (empty) ✓
 return

insertatbtm(temp); (1)

insertatbtm(int x): (d)

if (empty) ✓
 int t = pop(); t = 1
 insertatbtm

insertatbtm(int x): (e)

is (empty) ✓
 push(1);
 return

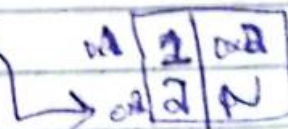
insertatbtm(temp); t = 2

insertatbtm(int x) (a.f)

if (empty) ✓
 int t = pop(); t = 1;
 call insertatbtm;

insertatbtm:
 is (empty)
 push(2);
 return

push(1);
 return;



Reverse List →

Question #04

```
void sortedinsert(int w)
```

```
{
    if (isep() || peek() < w)
```

```
    push(w);
    return;
```

```
}
```

```
int t = pop();
```

```
sortedinsert(w);
```

```
push(t);
```

```
}
```

// if val > top of stack

// push val

// return to caller

A store pop into t

// recursive call

// push t

```
void sortstack()
```

```
{
```

```
    if (isep())
```

```
    {
```

```
        return;
```

```
    }
```

```
    int t = pop();
```

```
    sortstack();
```

```
    sortedinsert(t);
```

```
}
```

// if empty

// return to caller

// store pop into t

// recursive call

// call function

(DRY RUN)

Suppose our stack is:

0x1	2	0x2
0x2	1	0x3
0x3	3	N

a) sortstack:

```
if (—) x
```

```
int t = pop() t = 2
```

```
sortstack();
```

```

sortstack(2):
if (-) x
int t = pop() t=1
sortstack:

```

```

sortstack(3):

```

```

if (-) x

```

```

int t = pop() t=3

```

```

sortstack:

```

stack [10]

```

sortstack(4):

```

```

if (-) v

```

```

return: (to sortstack3)

```

```

sortedingest(t): t=3

```

```

sorted:

```

```

if (ep ||) v

```

```

push(x);

```

```

return: (to stack2)

```

[3 | N]

```

sorted(t)

```

t=2

```

sorted:

```

```

if (-) v

```

```

push(x);

```

```

return: (to stack1)

```

[1 | 0 x 3]
[3 | N]

```

sorted(t)

```

t=2:

```

sorted:

```

```

if (-) x

```

```

int t = pop

```

```

sorted(t)

```

t=1

```

sorted(2):

```

```

if (-) v

```

```

push(t)

```

```

return:

```

[2
3]

push(t)

[1
2
3]

→ top sorted

Question # 05

(DRY_RUN)

Suppose: our stack is

arr	7	-
arr	4	-
arr	6	-

a) delete middle element:

if (-x)

int mid = size() / 2;

mid = 3 / 2 = 1

del helper(0, mid);

①
del helper(curr, mid)

curr = 0, mid = 1

if (-x)

0 != 1

int t = pop()

t = 7

del helper(curr + 1, mid);

②
del helper(curr, mid)

curr = 1, mid = 1

if (-x) ✓

pop()

4 (poped)

returns (to dh 1)

push(temp);

7
6

Question # 06

(DRY_RUN)

Suppose: our expression is { () }

is balanced

Stack S;

4
7

1) for (int i=0; i<str.length(); i++) ✓

ch = str[i];

ch = {

if (ch == '(') ✓
{ s.push(ch); }
else if (—) ✗

{	N
---	---

2) for (—) ✓ 1 < 4
ch = str[i];
if (ch == ')') ✓
{ s.push(ch); }
else if (—) ✗

ch = {

{	N
{	N

3) for (—) ✓ 2 < 4
ch = str[i];
if (—) ✗
else if (—) ✓
{

ch =)

if (—) ✗
char topch = s.pop();
if (—) ✗

topch = (

Stack → { | N

4) for (—) ✓ 3 < 4
ch = str[i];
if (—) ✗
else (—) ✓

ch = }

{ if (—) ✗
char topch = s.pop();
if (—) ✗

topch = {

Question # 007

(DRY_RUN)

Suppose $n = 4$

arr = [1, 2, 3, 4]
ans = []

a) next greater element (ind arr[], ind n):

Stack S

int result[n];

a) for(int i: n-1; i >= 0; i--) ✓ $i >= 0$

while(!s.isempty() && s.peek() < arr[i])

if(s.isempty()) ✓

res[i] = -1;

else (→) ✓

s.push(arr[i])

→ arr[i] = 3

res[i] = -1

3

a) for (→) ✓ $i >= 0$

while(!s.isempty() && s.peek() < arr[i]) ✓

s.pop()

3 popped

if(s.isempty()) ✓

res[i] = -1;

else (→) ✓

s.push(arr[i]);

← arr[i] = 4

res[i] = { ..., -1, -1 }

4

a) for (→) ✓ $i >= 0$

```

while ( ) {
    if ( ) {
    } else {
        res[i] = s.peek();
    }
}

```

res = {4, -1, -1}

s.push(xyz); ?

if () { } > 20

Question # 08

(DRY_RUN)

Suppose stack is:

bool isFull()

{

if () {

Node* front = top;

(Remove)

{f > 0x1}

return cph(top, front);

0x1	1	→ top,
0x2	2	front
0x3	1	

① bool cph(Node* cur, Node* F) cur = 0x1, F = 0x1

if () {

bool res = cph(c->next, F);

②

bool cph(Node* c, Node* F) c = 0x2, F = 0x1

if () {

bool res = cph(c->next, F);

③

bool cph(Node* c, Node* F) c = 0x3, F = 0x1

if () {

bool res = cph(c->next, F);

if (cph(N) < 0)
if (—) ✓
return true;
// return to ②

if (—) ✗
bool checkz (curr->d == f->d) {

front = front->next; f = 0;
return check;
// return to b

if (—) ✗ checkz true

bool checkz (c->d == f->d); c = 0, f = 0
222 ✓

front = front->next;

↓
continue

if's palindromic